TALLER DE METODOLOGÍAS ÁGILES · 2026-2 · TERCER CORTE

Punto de control del proyecto final

Programar en pareja con una persona o con un agente: qué se gana, qué se pierde y qué aprendizaje desaparece

TEMA DE EXPOSICIÓN

3 · Pair programming

PRÁCTICA QUE ASISTE EL AGENTE

Desarrollo de una tarea con el agente como copiloto

PAREJA

Ana Sofía Arango Yanza · Juan José Arias Chacua

PROFESOR

César Jesús Pardo Calvache, Ph.D.

FECHA Y VERSIÓN

Viernes 30 de octubre de 2026 · v2 (29/10; reemplaza el borrador v1 del 26/10)

REPOSITORIO

github.com/Sofii141/CAMEIA-lite · ramas `main` (código) y `documentacion`

0 Estado frente a lo que pide el punto de control

El punto de control revisa tres cosas: el caso base, la especificación del agente (v1) y la línea base ya medida. Además, ese día se intercambian tres casos con otra pareja.

CASO BASE

24 casos

11 S · 11 M · 2 L · **10 difíciles**

ESPECIFICACIÓN V1

1 agente

8 campos · herramienta y modelo se fijan antes del ciclo 1

LÍNEA BASE

10 tareas

pareja humana, sin IA · medida el 29/10 (§5)

Requisito del enunciado	Dónde está	Estado
Caso base con ≥ 10 casos que incluye material difícil	§2 · repo `main` (`TAREAS.md`)	Listo
Caso congelado antes de tocar el agente	commit `congelado v1` en `main`	Antes de medir
Línea base medida con GQM, ≥ 10 casos y condiciones registradas	§3, §5 y bitácora v2 (anexo)	29/10
Especificación de agentes v1 con sus 8 campos	§7 · `04 - Especificacion agentes/29102026_01_Spec_Copiloto_v1.md`	Lista · falta la herramienta
Tres casos para entregar a otra pareja y tres que recibimos	`02 - Caso base/Casos entregados` y `Casos recibidos`	30/10

REGLA QUE SEGUIMOS

La línea base (§5) se llena **solo** con lo medido en las sesiones cronometradas y grabadas: el script `07 - Analisis/Scripts/resumen_medicion.py` la copia aquí desde la bitácora. Si no alcanzamos las 10 tareas, se reporta el n real con la marca `[N]` .

1 Tema, práctica e hipótesis

El enunciado asigna al tema 3 la práctica **"desarrollo de una tarea con el agente como copiloto"** y sugiere medir tiempo, retrabajo y errores detectados a tiempo. Para que el proyecto y la exposición cuenten una sola historia, agregamos la pregunta central del tema: **qué aprendizaje se pierde** cuando el compañero es un agente que no necesita entender.

Qué dice la literatura revisada por pares (y qué esperamos ver)

Hallazgo publicado	Hipótesis para nuestro caso
Hannay et al. 2009 (metaanálisis, *IST*) y **Arisholm et al. 2007** (*IEEE TSE*, 295 profesionales): en general, la pareja no reduce el tiempo, cuesta un 84 % más de esfuerzo y su efecto depende de la complejidad.	**H1:** con el agente la duración baja en las tareas S y M, pero no en las difíciles.
Perry et al. 2023 (*ACM CCS*): con IA el código fue menos seguro, y aun así quienes la usaron creían que era más seguro. (Contexto: el preprint de METR 2025 encontró un 19 % más de lentitud, aunque esperaban ir un 24 % más rápido.)	**H2:** la ayuda percibida con el agente supera lo que muestra el cronómetro (real/pronóstico).
Welter et al. 2025 (*ASE*): las sugerencias de Copilot se aceptan con menos escrutinio que las de un compañero humano.	**H3:** con el agente se escapan más errores a las pruebas ocultas, sobre todo en las trampas de seguridad (t18–t20).
Plonka et al. 2015 (*IJHCS*) y **Shihab et al. 2025** (*ICER*): la pareja transfiere conocimiento; con Copilot, los estudiantes no entendían por qué funcionaban las sugerencias. Contrapeso: **Kazemitabaar et al. 2023** (*CHI*) no encontró menos retención.	**H4:** el codewalk baja con el agente, sobre todo el del compañero que no participó (transferencia de conocimiento).

EL ESTUDIO MÁS PARECIDO AL NUESTRO

Fan et al. 2025 (*International Journal of STEM Education*) comparó pareja con IA, pareja humana e individual con 234 estudiantes. Con IA hubo mejor desempeño en las tareas, pero la pareja humana tuvo la mayor percepción de colaboración y de presencia social. Nosotros miramos lo que ese estudio no mide: tiempo, errores escapados y comprensión del código, tarea por tarea.

CÓMO LEEREMOS LOS RESULTADOS

Con unas 10 tareas por condición **no** podemos afirmar causalidad. Reportaremos medianas, rangos y tamaño de efecto, y diremos qué **no** demuestra cada número (§6). Si el agente no mejora la práctica, lo diremos con los datos.

2 El caso base: CAMEIA-lite

24 tareas pequeñas de programación. Cada una aísla una regla **real** del backlog de CAMEIA, el proyecto en el que trabajamos los dos durante el semestre.

De dónde sale

De los PR fusionados en los 9 repositorios de CAMEIA (324, extraídos con `gh` el 25/10). Cada tarea viene de un ítem de Jira (17 en total, en 5 repos) y de su PR. La trazabilidad completa está en `02 - Caso base/29102026_01_Backlog_Origen_CAMEIA-lite_v2.xlsx`.

Por qué así y no directamente sobre CAMEIA

Las tareas reales exigen montar Docker, Firebase y PostgreSQL, y sus PR son muy desiguales. El cronómetro mediría la configuración y no la práctica. CAMEIA-lite conserva los requisitos reales, se corre con un solo comando y permite tareas comparables entre condiciones.

Qué lo hace representativo y difícil

- **11 S · 11 M · 2 L**, asignadas por sorteo estratificado (§4).
- **10 difíciles**: Unicode/NFC, 29 de febrero, concurrencia por doble clic, orden de reglas 404/409, inyección de cabeceras, el subdominio trampa en CORS, la variable vacía que sí existe y el falso obvio `production` / `prod`.
- Cada tarea trae **pruebas visibles** (39, guían el desarrollo) y **pruebas ocultas** (71, miden los errores escapados).
- Las 110 pruebas fallan con el esqueleto vacío (comprobado el 29/10).

Las 24 tareas

ID	Tarea	Tam.	Difícil	Origen (Jira)	Ya la hizo
`t01`	Normalizar texto	S		CM-36	—
`t02`	Contar caracteres	S		CM-36	—
`t03`	Validar nombre	M	Difícil	CM-36	—
`t04`	Validar correo	S		CM-36	—
`t05`	Validar celular	S		CM-36	—
`t06`	Política de contraseñas	M		CM-36	—
`t07`	Fecha de nacimiento	M	Difícil	CM-279	—
`t08`	Cupo de perfiles por plan	M	Difícil	CM-271	—
`t09`	Nivel de log por estado HTTP	S		CM-184	—

ID	Tarea	Tam.	Difícil	Origen (Jira)	Ya la hizo
t10	Mensajes de error por campo	M		CM-336 / CM-298	Juan José
t11	Contador de reintentos	M		CM-217	Juan José
t12	Crear una sola vez (doble clic)	L	Difícil	CM-270	Juan José
t13	Eliminar rol objetivo	M	Difícil	CM-71 / CM-271	—
t14	Sustituir rol conservando su lugar	S		CM-72	—
t15	Catálogo de roles por idioma	S		CM-82 / CM-72	Ana Sofía
t16	Fechas de formación	M		CM-271	—
t17	Expectativa salarial	S		CM-271	—
t18	Propagar X-Request-Id	M	Difícil	CM-279 / CM-104	—
t19	Limpiar cabeceras de identidad	M	Difícil	CM-104	—
t20	Origen permitido (CORS)	S	Difícil	CM-162	—
t21	Perfil de arranque y emulador	M	Difícil	CM-190 / CM-191 / CM-175	—
t22	Registro idempotente (Idempotency-Key)	L	Difícil	CM-340	—
t23	Progreso de la entrevista	S		CM-217	Juan José
t24	Enmascarar correo en logs	S		CM-271 (registro sin datos personales)	—

LIMITACIONES DECLARADAS

- Las reglas se sacan de su entorno, así que no medimos el costo de integrarlas con el sistema real.
- Los PR originales se escribieron con ayuda de Claude Code, así que la solución de referencia puede tener "estilo de agente".
- Las tareas y sus pruebas se generaron con Claude Code, a cargo de Ana Sofía: ella conoce el diseño del caso. Lo declaramos como amenaza interna.
- Cinco tareas ya las implementó uno de nosotros en CAMEIA: van solo en la condición con agente y las hace la otra persona.
- Cinco reglas se ajustaron para hacer el caso más difícil; van marcadas `[P]` en el backlog de origen.

3 GQM

Objetivo: analizar el desarrollo de una tarea con un agente copiloto, frente al pair programming humano, para evaluar su efecto en tiempo, esfuerzo, errores escapados, retrabajo y comprensión del código, en las 24 tareas de CAMEIA-lite.

Pregunta	Métrica	Cómo se toma	Qué NO representa	Respaldo
Q1 ¿Se termina más rápido?	M1 Duración (min)	Cronómetro desde abrir el enunciado hasta pasar las pruebas visibles, menos pausas	Productividad general	Hannay 2009; Shihab 2025
Q2 ¿Cuesta más persona-tiempo?	M2 Esfuerzo = M1 × personas	Calculado (pareja = 2; persona + agente = 1)	Tiempo de aprender la herramienta. Favorece al agente por diseño	Arisholm 2007
Q3 ¿Se escapan más errores?	M3 Ocultas que fallan / total	`pytest tests/ocultas -k tNN` al declarar "terminado"	Defectos de diseño o de legibilidad	Arisholm 2007; Dakhel 2023
Q4 ¿Hay más retrabajo?	M4 Minutos y commits después de "terminado"	Cronómetro aparte + `git log`	El tamaño de cada corrección	Enunciado del curso
Q5 ¿Qué aprendizaje se pierde?	M5 Codewalk 0–5 (autor y compañero)	5 preguntas; quien responde no mira el código y quien evalúa sí	La habilidad general	Plonka 2015; Welter 2025
Q6 ¿Nos engaña la sensación de velocidad?	M6 Pronóstico antes, ayuda percibida después; real/pronóstico	Hoja de medición	El resultado real	Perry 2023; Lee 2025

Las 5 preguntas del codewalk (1 punto cada una, máximo 5 min por persona)

1. ¿Qué hace la función, en una frase?
2. ¿Qué caso borde maneja y dónde?
3. ¿Por qué se eligió esta forma y no otra?
4. Si mañana falla con la entrada X, ¿dónde mirarías primero? *(depuración; X la escoge el evaluador)*
5. ¿Qué cambiarías si el requisito cambia en Y? *(Y la escoge el evaluador)*

4 Diseño de la medición

	Línea base (29/10)	Ciclo 1 (2–6 nov)	Ciclo 2 (9–13 nov)
Condición	Pareja humana: conductor y navegante, cambian cada 15 min, sin IA	Una persona + el agente copiloto; el compañero no participa	
Tareas	10 por sorteo: **5 S · 4 M · 1 L**	12 por sorteo (6 + 6): **5 S · 6 M · 1 L**, más los 3 casos recibidos en el ciclo 2	
Evidencia	Grabación de pantalla, commits por tarea, hoja de medición, hoja de sesiones y, en los ciclos, una fila de bitácora por cada uso del agente		

Protocolo por tarea

1. Anotar el pronóstico de minutos.
2. Iniciar el cronómetro al abrir el enunciado y hacer commit `tNN: inicio`.
3. Trabajar solo con las pruebas visibles (`python -m pytest -k tNN`; por defecto ya no corre las ocultas).
4. Declarar "terminado" cuando pasan, parar el cronómetro y hacer commit `tNN: terminado`.
5. Correr las pruebas ocultas: cada fallo es un error escapado. Si se corrige, cuenta como retrabajo (`tNN: retrabajo`).
6. Codewalk: máximo 5 minutos por persona.
7. Llenar la hoja de medición en ese momento, no al final del día.

CONTROLES PARA QUE LA COMPARACIÓN VALGA

- **Sorteo reproducible y estratificado** (`python sorteo.py <semilla>`), con cupos fijos por tamaño. Las difíciles se reparten alternando entre condiciones y **el orden de las tareas también se sortea**. La semilla y la salida quedan en `registro/`.
- Corregimos el sorteo el 29/10, **antes de usarlo**: la versión anterior dejaba siempre la línea base con 7 S y solo 2 M, es decir, más fácil que la condición con agente.
- Si alguien abre las **pruebas ocultas antes de tiempo**, la tarea se descarta con la marca `[C]` y se usa una de repuesto.
- En los ciclos, el agente trabaja en una copia del repo **sin** `tests/ocultas`.

5 Línea base

PENDIENTE DE MEDIR

Esta sección la llena `07 - Analisis/Scripts/resumen_medicion.py` a partir de la bitácora v2, después de las sesiones del 29/10. No se escribe a mano.

6 Cómo presentaremos los resultados

Con pocas tareas por condición, mostramos todos los datos y no solo un promedio.

Decisión	Por qué	Fuente
Mediana [mínimo–máximo] por tamaño, en lugar del promedio	Una sola tarea larga no arrastra el resultado	Kitchenham et al. 2017, *EMSE*
A12 como tamaño de efecto (0,5 = iguales)	Es la probabilidad de que una tarea con agente tarde más; se entiende sin estadística	Vargha & Delaney 2000, *JEBS*
Sin p-valores ni causalidad	Con n ≈ 10 no hay potencia estadística: el análisis es descriptivo	Kitchenham et al. 2002, *IEEE TSE*
Gráfico de puntos: una fila por tamaño y un punto por tarea	Se ve cada tarea, no solo el resumen	Kitchenham et al. 2017
Recálculo independiente: script, hoja de cálculo y la otra persona a mano	Fue lo que nos faltó en la E1	—
Amenazas a la validez en cada conclusión	Aprendizaje entre semanas, 2 personas frente a 1, tareas aisladas	Wohlin et al. 2012; Ko et al. 2015, *EMSE*

7 Especificación del agente • v1

Campo	Contenido
Rol y alcance	Hace de navegante en el desarrollo de *una* tarea de CAMEIA-lite: propone un enfoque, el código y los casos borde que conviene probar. La persona conduce y decide.
Entradas	Solo el enunciado y los criterios de aceptación de la tarea (`TAREAS.md`), su archivo `src/cameia_lite/tNN.py` y sus pruebas visibles.
Salidas	En markdown fijo: un plan de 3 a 6 pasos, el diff (solo en `tNN.py`), los supuestos que hizo y los casos borde que cree que faltan, como texto.
Herramientas y permisos	Herramienta y modelo exactos: **se fijan antes del 2/11** . Trabaja en una copia del repo **sin** `tests/ocultas` . Sin acceso al remoto ni a credenciales.
Prohibiciones	No hacer commit, push ni merge. No modificar `tests/` ni otros archivos. No leer las pruebas ocultas. No desactivar ni saltarse pruebas. No inventar requisitos. No manejar credenciales ni datos personales.
Criterio de validación	Se acepta solo si pasan las pruebas visibles, cumple cada criterio de aceptación y quien conduce puede explicar cada línea sin mirar al agente.
Punto de control humano	Antes de cada commit, quien conduce decide aceptar, corregir o rechazar y lo registra en la bitácora con su razón. Las pruebas ocultas las corre la persona. Después viene el codewalk con el compañero.
Versión	v1 · 29/10/2026. Cada cambio da una versión nueva, con el dato de la bitácora que la motivó.

Empezamos con un solo agente. Solo agregaremos un segundo ("revisor") en el ciclo 2 si la bitácora del ciclo 1 muestra un problema que el primero no resuelve.

8 Calendario y preguntas

29 oct
Línea base: 10 tareas en pareja humana

30 oct
Punto de control e intercambio de 3 casos

2–6 nov
Ciclo 1 con el agente, bitácora y spec v2

9–13 nov
Ciclo 2, 3 fallos documentados y video

16–18 nov
Reporte, diapositivas y exposición

Preguntas para el profesor

☐ ¿Acepta CAMEIA-lite (requisitos reales de CAMEIA aislados como funciones) como caso base, en lugar de trabajar directamente sobre los repositorios?

☐ ¿Es válido agregar la comprensión del código (codewalk) a las métricas sugeridas para el tema 3?

☐ ¿Con qué pareja intercambiamos los tres casos, y en qué formato? Proponemos una ficha con enunciado, criterios de aceptación y pruebas.

9 Fuentes

DOI verificados en Crossref y OpenAlex el 29/10/2026. Lista completa, con base de datos y estado de lectura: `01 - Exposicion/Fuentes/29102026_01_Fuentes_v2.md`.

Revisadas por pares

- Arisholm, E., Gallis, H., Dybå, T., & Sjøberg, D. I. K. (2007). Evaluating pair programming with respect to system complexity and programmer expertise. *IEEE Transactions on Software Engineering, 33*(2), 65–86. doi:10.1109/TSE.2007.17
- Fan, G., Liu, D., Zhang, R., & Pan, L. (2025). The impact of AI-assisted pair programming on student motivation, programming anxiety, collaborative learning, and programming performance. *International Journal of STEM Education, 12*, 16. doi:10.1186/s40594-025-00537-3
- Hannay, J. E., Dybå, T., Arisholm, E., & Sjøberg, D. I. K. (2009). The effectiveness of pair programming: A meta-analysis. *Information and Software Technology, 51*(7), 1110–1122. doi:10.1016/j.infsof.2009.02.001
- Kazemitabaar, M., Chow, J., Ma, C. K. T., Ericson, B. J., Weintrop, D., & Grossman, T. (2023). Studying the effect of AI code generators on supporting novice learners in introductory programming. *CHI 2023*, 1–23. doi:10.1145/3544548.3580919
- Kitchenham, B. et al. (2002). Preliminary guidelines for empirical research in software engineering. *IEEE TSE, 28*(8), 721–734. doi:10.1109/TSE.2002.1027796
- Kitchenham, B., Madeyski, L., Budgen, D., Keung, J., Brereton, P., Charters, S., Gibbs, S., & Pohthong, A. (2017). Robust statistical methods for empirical software engineering. *Empirical Software Engineering, 22*(2), 579–630. doi:10.1007/s10664-016-9437-5
- Ko, A. J., LaToza, T. D., & Burnett, M. M. (2015). A practical guide to controlled experiments of software engineering tools with human participants. *Empirical Software Engineering, 20*(1), 110–141. doi:10.1007/s10664-013-9279-3
- Lee, H.-P. et al. (2025). The impact of generative AI on critical thinking. *CHI 2025*, 1–22. doi:10.1145/3706598.3713778
- Moradi Dakhel, A. et al. (2023). GitHub Copilot AI pair programmer: Asset or liability? *Journal of Systems and Software, 203*, 111734. doi:10.1016/j.jss.2023.111734
- Perry, N., Srivastava, M., Kumar, D., & Boneh, D. (2023). Do users write more insecure code with AI assistants? *ACM CCS 2023*, 2785–2799. doi:10.1145/3576915.3623157
- Plonka, L., Sharp, H., van der Linden, J., & Dittrich, Y. (2015). Knowledge transfer in pair programming: An in-depth analysis. *International Journal of Human-Computer Studies, 73*, 66–78. doi:10.1016/j.ijhcs.2014.09.001
- Shihab, M. I. H., Hundhausen, C., Tariq, A., Haque, S., Qiao, Y., & Mulanda, B. (2025). The effects of GitHub Copilot on computing students' programming effectiveness, efficiency, and processes in brownfield coding tasks. *ICER 2025*, 407–420. doi:10.1145/3702652.3744219
- Vargha, A., & Delaney, H. D. (2000). A critique and improvement of the CL common language effect size statistics of McGraw and Wong. *Journal of Educational and Behavioral Statistics, 25*(2), 101–132. doi:10.3102/10769986025002101
- Welter, A., Schneider, N., Dick, T., Weis, K., Tinnes, C., & Wyrich, M. (2025). An empirical study of knowledge transfer in AI pair programming. *ASE 2025*, 166–177. doi:10.1109/ASE63991.2025.00022
- Wohlin, C., Runeson, P., Höst, M., Ohlsson, M. C., Regnell, B., & Wesslén, A. (2012). *Experimentation in software engineering*. Springer. doi:10.1007/978-3-642-29044-2
- Basili, V., Caldiera, G., & Rombach, H. D. (1994). The Goal Question Metric approach. En *Encyclopedia of Software Engineering*. Wiley.

Literatura gris (no revisada por pares; solo como contexto)

- Becker, J., Rush, N., Barnes, B., & Rein, D. (2025). Measuring the impact of early-2025 AI on experienced open-source developer productivity. METR, preprint arXiv:2507.09089.

10 Declaración de uso de IA

Herramienta	Para qué	Qué verificamos nosotros
Claude Code	Leer los enunciados y la retroalimentación de la E1; proponer la estructura; generar las 24 tareas con sus pruebas; armar el backlog de origen a partir de los PR; buscar fuentes y verificar sus DOI en Crossref y OpenAlex; revisar la coherencia de los documentos con un segundo agente revisor; maquetar este documento.	**Completar antes de entregar** que revisamos las 24 tareas, que cada fila del backlog coincide con su PR y qué fuentes leímos (las que tengan `[V]`).

Ningún dato de línea base ni de bitácora fue generado por IA: esos datos salen solo de nuestras sesiones cronometradas y grabadas.